

TALLER PRÁCTICO – BLOQUE 5

De clases aisladas a objetos que colaboran

Interfaces · Asociación · Agregación · Composición · Herencia ·
Componentes UML

Caso conductor: SmartLibrary

Estudiante: Cristhian Santiago Parra Erazo

ID: 926222

Documento de desarrollo y justificación

Universidad Cooperativa de Colombia– Campus Pasto

1. Actividad 1 – Diagnóstico de relaciones

Objetivo. Analizar el significado de las relaciones del dominio antes de seleccionar el símbolo UML, considerando si los objetos pueden existir independientemente.

1.1 Diagnóstico inicial

Elementos	¿Qué relación existe en el problema?	¿Existencia independiente?	Decisión inicial
Estudiante – Préstamo	El estudiante participa como responsable de un préstamo.	Sí. El estudiante existe antes, durante y después del préstamo.	Asociación. La relación expresa colaboración entre dos objetos independientes.
Libro – Ejemplar	Un libro puede tener varios ejemplares físicos y cada ejemplar corresponde a un único libro.	Sí, bajo el supuesto de que el ejemplar como recurso físico mantiene su ciclo de vida propio.	Agregación. Existe una relación todo–parte, pero el ejemplar conserva independencia conceptual y física.
Préstamo – Renovación	Una renovación conserva información histórica de un cambio realizado sobre un préstamo.	No. La renovación se define como parte del historial del préstamo.	Composición. La renovación carece de sentido dentro del dominio si se separa del préstamo al que pertenece.
Usuario – Estudiante	Estudiante representa una especialización de Usuario.	Sí. Un estudiante es un usuario específico del sistema.	Generalización / herencia. La relación corresponde a un “es-un”.
Usuario – Bibliotecario	Bibliotecario representa otra especialización de Usuario.	Sí. El bibliotecario es un tipo concreto de usuario.	Generalización / herencia. Se comparte una abstracción común y cada especialización mantiene información particular.

Cuadro 1: Diagnóstico inicial de las relaciones del dominio.

1.2 Preguntas de discusión

1. ¿Que dos clases se relacionen significa necesariamente que una sea parte de la otra?

No. Que dos clases estén relacionadas únicamente significa que existe alguna forma de

interacción, conocimiento o dependencia conceptual entre ellas. Una asociación puede representar colaboración sin que exista una relación todo–parte. Por ello, no se debe asumir automáticamente agregación o composición solo porque exista una relación entre dos clases.

2. ¿Qué evidencia del requisito se utilizó para decidir?

La principal evidencia está en los requisitos R10 a R13. R10 establece que un libro puede tener varios ejemplares y que cada ejemplar corresponde a un único libro; R11 indica que un préstamo registra sus renovaciones; R12 establece la existencia de usuarios, estudiantes y bibliotecarios; y R13 indica que algunos usuarios pueden recibir notificaciones.

3. ¿Cuál de las relaciones anteriores sería incorrecto decidir únicamente mirando los nombres de las clases?

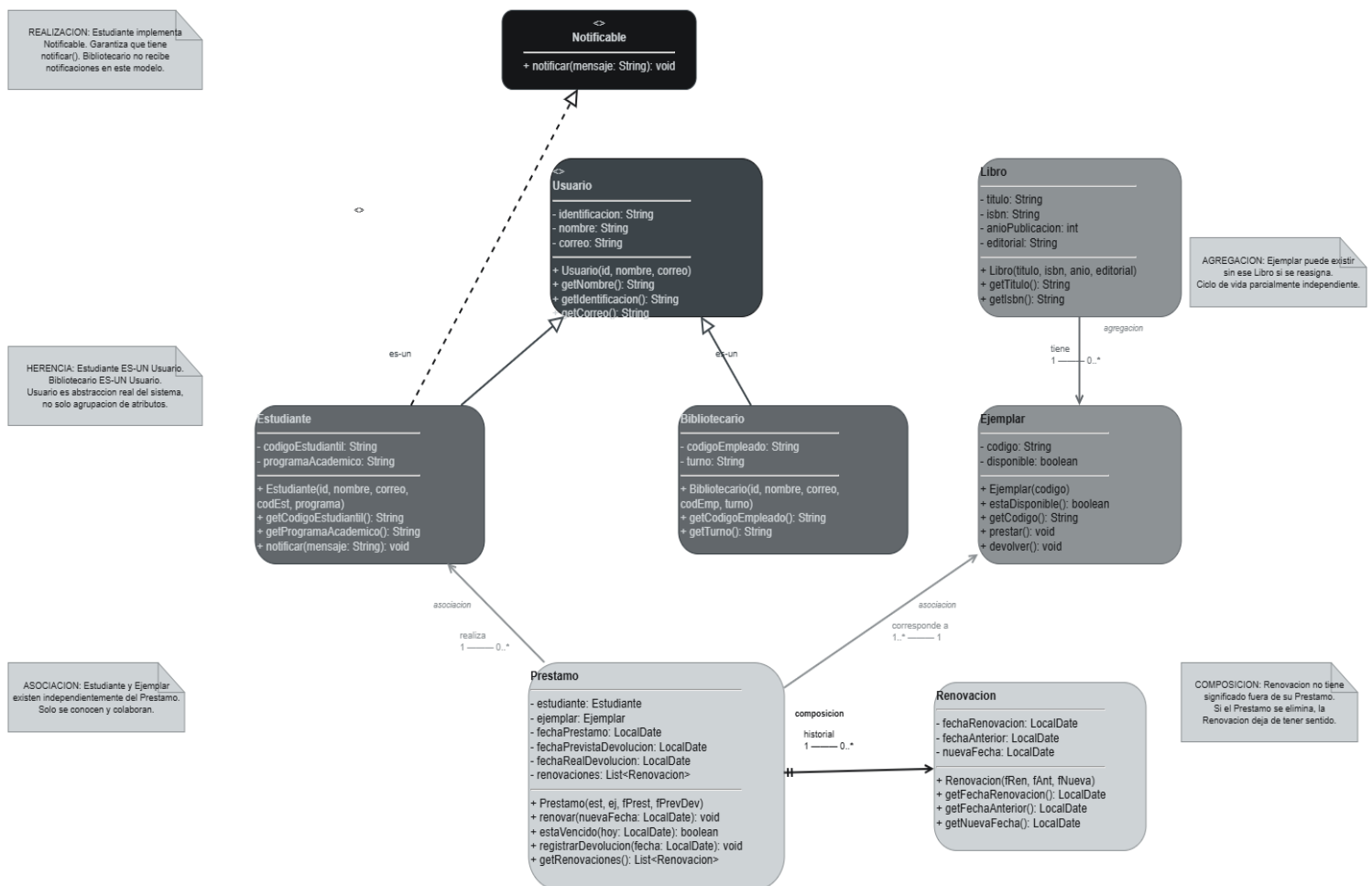
La relación **Libro–Ejemplar** es la que requiere mayor análisis, porque el nombre “Ejemplar” puede sugerir automáticamente una composición. La decisión correcta depende del supuesto sobre el ciclo de vida del ejemplar. Por ello, debe justificarse con el significado del dominio y no solamente por el nombre de las clases.

2. Actividad 2 – Asociación, agregación y composición

Objetivo. Construir el fragmento del diagrama de clases incluyendo **Libro**, **Ejemplar**, **Estudiante**, **Prestamo** y **Renovacion**, con multiplicidades y tipos de relación coherentes con el dominio.

2.1 Diagrama de clases

Diagrama de Clases — SmartLibrary (Bloque 5)



2.2 Justificación de las cuatro relaciones

Relación	Tipo seleccionado	Justificación basada en el dominio
Estudiante – Préstamo	Asociación	El estudiante participa en uno o varios préstamos a lo largo del tiempo, mientras que cada préstamo corresponde a un estudiante. Ambos objetos tienen existencia conceptual independiente.
Préstamo – Ejemplar	Asociación	El préstamo necesita identificar el ejemplar que se presta, pero ni el préstamo ni el ejemplar dependen conceptualmente del otro para existir. La relación representa colaboración dentro de una operación del sistema.
Libro – Ejemplar	Agregación	R10 establece que un libro puede tener varios ejemplares y que cada ejemplar corresponde a un único libro. Se asume que el ejemplar posee un ciclo de vida independiente como recurso físico; por eso se representa como agregación y no como composición.
Préstamo – Renovación	Composición	R11 indica que la renovación conserva el historial de modificaciones de un préstamo. La clase Renovación se modela como parte interna de ese historial: sin el préstamo correspondiente, la renovación pierde su significado dentro del dominio.

Cuadro 2: Tipos de relación y justificación.

2.3 Criterio de decisión

Las relaciones se seleccionaron según tres preguntas: si las clases solamente necesitan conocerse o colaborar, si existe una relación conceptual todo–parte y, en ese caso, si la parte conserva un ciclo de vida independiente. Este criterio evita utilizar composición de manera automática.

3. Actividad 3 – Herencia: ¿es-un o solo se parecen?

Objetivo. Evaluar si la generalización **Usuario–Estudiante– Bibliotecario** representa realmente una relación “es-un”.

3.1 Decisión de generalización

4. ¿Estudiante es realmente un tipo de Usuario dentro de SmartLibrary?

Sí. R12 establece que el sistema maneja usuarios de biblioteca y que los estudiantes y bibliotecarios comparten información de identificación, nombre y correo, además de poseer información particular. En consecuencia, un **Estudiante** puede modelarse como una especialización concreta de **Usuario**.

5. ¿Bibliotecario es realmente un tipo de Usuario?

Sí. El bibliotecario también pertenece al conjunto de usuarios gestionados por SmartLibrary y comparte la información común definida para Usuario, pero añade atributos propios como *codigoEmpleado* y *turno*. Por tanto, la relación “Bibliotecario es un Usuario” representa adecuadamente una generalización.

6. ¿La existencia de identificacion, nombre y correo repetidos sería suficiente para crear la superclase Usuario? ¿Por qué?

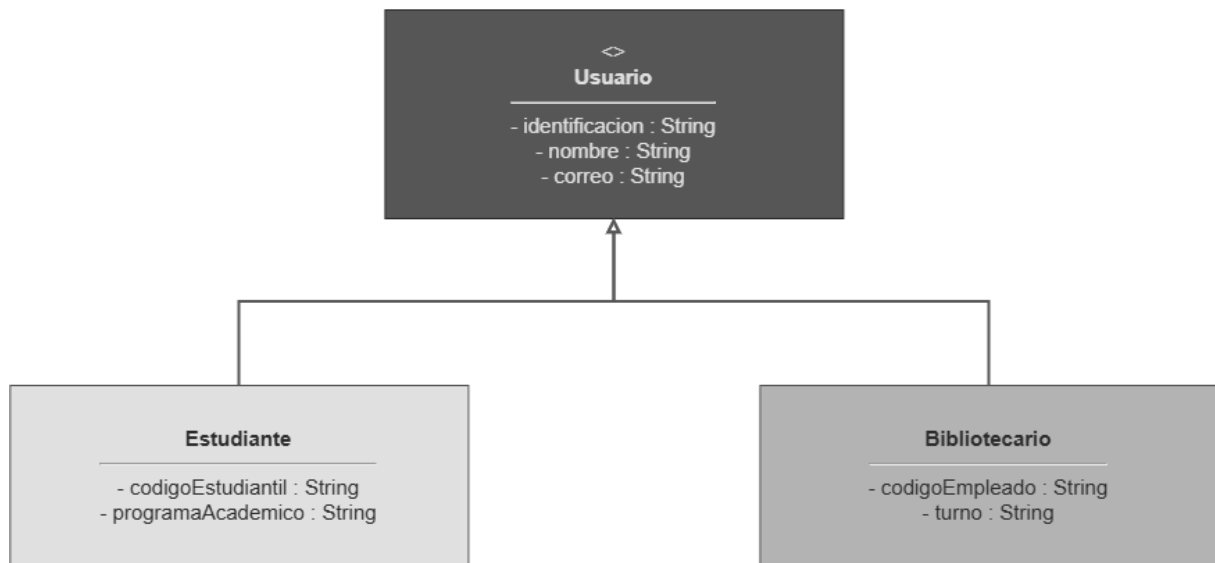
No. La repetición de atributos, por sí sola, no es evidencia suficiente. La herencia debe expresar una relación semántica de tipo “es-un” y una abstracción real dentro del dominio. En SmartLibrary, además de compartir atributos, Estudiante y Bibliotecario representan tipos concretos de usuarios del sistema, por lo que la generalización tiene significado conceptual.

7. Situación donde crear una superclase solo para evitar repetición sería débil.

Sería débil crear una superclase únicamente porque dos clases poseen *nombre* y *correo*, cuando en realidad pertenecen a conceptos distintos, poseen reglas diferentes y no representan especializaciones de una misma abstracción del dominio. En ese caso, la herencia resolvería duplicación de código, pero no expresaría correctamente el significado del negocio.

3.2 Diagrama actualizado

Diagrama UML 3.2 - Generalización de Usuario



4. Actividad 4 – Interfaces: expresar un contrato

Objetivo. Expresar mediante la interfaz **Notificable** un contrato de comportamiento para los objetos que pueden recibir notificaciones.

4.1 Decisión sobre las clases notificables

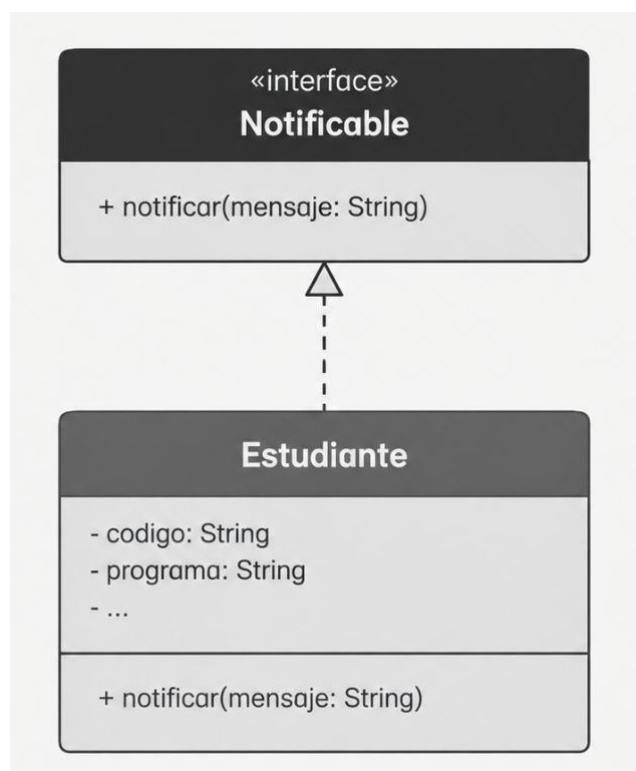
Se decide que **Estudiante** implemente **Notificable**. Esta decisión se relaciona con el funcionamiento del préstamo: después de una renovación, el sistema puede notificar al estudiante sobre el cambio realizado.

Para **Bibliotecario**, el requisito R13 indica que “algunos usuarios” pueden recibir notificaciones, pero no obliga a que todos los tipos de usuario sean notificables. Por ello, no se añade la interfaz a Bibliotecario sin una regla adicional del dominio que lo justifique.

4.2 Interfaz propuesta

```
public interface Notificable {  
    void notificar(String mensaje);  
}
```

4.3 Representación UML



4.4 Respuestas del contrato

Pregunta	Respuesta
¿Qué promete una clase que implementa Notificable?	Promete proporcionar una implementación del comportamiento <code>notificar(String mensaje)</code> definido por el contrato.
¿La interfaz define cómo se envía el mensaje?	No. Define qué operación debe existir, pero no especifica la estrategia, medio o mecanismo utilizado para enviar la notificación.
¿Puede una clase heredar de Usuario e implementar Notificable al mismo tiempo?	Sí. Una clase puede extender una clase y, simultáneamente, implementar una interfaz. En este caso, Estudiante puede heredar de Usuario y cumplir el contrato Notificable.

Cuadro 3: Preguntas sobre el contrato Notificable.

4.5 Diferencia entre interfaz y superclase

Una interfaz representa principalmente un **contrato de comportamiento**: indica qué operación debe estar disponible. Una superclase representa una **abstracción de tipo** de la cual una subclase hereda estructura y comportamiento definidos por esa jerarquía. Por ello, implementar una interfaz no equivale a heredar de una superclase.

5. Actividad 5 – Implementación representativa en Java

Objetivo. Implementar únicamente el fragmento necesario para validar las decisiones de diseño, sin construir una aplicación completa.

5.1 Usuario, Estudiante y Bibliotecario

```
public abstract class Usuario {  
  
    private String identificacion;  
    private String nombre;  
    private String correo;  
  
    public Usuario(String identificacion, String nombre, String correo) {  
        this.identificacion = identificacion;  
        this.nombre = nombre;  
        this.correo = correo;  
    }  
  
    public String getIdentificacion() { return identificacion; }  
    public String getNombre()         { return nombre; }  
    public String getCorreo()          { return correo; }  
}
```

```
public class Estudiante extends Usuario implements Notificable {

    private String codigoEstudiantil;
    private String programaAcademico;

    public Estudiante(
        String identificacion,
        String nombre,
        String correo,
        String codigoEstudiantil,
        String programaAcademico) {

        // Llama al constructor de Usuario con los atributos compartidos
        super(identificacion, nombre, correo);
        this.codigoEstudiantil = codigoEstudiantil;
        this.programaAcademico = programaAcademico;
    }

    public String getCodigoEstudiantil() { return codigoEstudiantil; }
    public String getProgramaAcademico() { return programaAcademico; }

    @Override
    public void notificar(String mensaje) {
        System.out.println("[NOTIFICACION para " + getNombre() + "] " + mensaje);
    }
}
```

```
public class Bibliotecario extends Usuario {

    private String codigoEmpleado;
    private String turno;

    public Bibliotecario(
        String identificacion,
        String nombre,
        String correo,
        String codigoEmpleado,
        String turno) {

        super(identificacion, nombre, correo);
        this.codigoEmpleado = codigoEmpleado;
        this.turno = turno;
    }

    public String getCodigoEmpleado() { return codigoEmpleado; }
    public String getTurno() { return turno; }
}
```

5.2 Renovacion y Prestamo

```
public class Renovacion {  
  
    private LocalDate fechaRenovacion; // cuando se realizo la renovacion  
    private LocalDate fechaAnterior;   // fecha prevista que habia antes  
    private LocalDate nuevaFecha;      // nueva fecha prevista de devolucion  
  
    public Renovacion(  
        LocalDate fechaRenovacion,  
        LocalDate fechaAnterior,  
        LocalDate nuevaFecha) {  
  
        this.fechaRenovacion = fechaRenovacion;  
        this.fechaAnterior   = fechaAnterior;  
        this.nuevaFecha      = nuevaFecha;  
    }  
  
    public LocalDate getFechaRenovacion() { return fechaRenovacion; }  
    public LocalDate getFechaAnterior()   { return fechaAnterior; }  
    public LocalDate getNuevaFecha()      { return nuevaFecha; }  
  
    @Override  
    public String toString() {  
        return "Renovacion["  
            + "realizada=" + fechaRenovacion  
            + ", fechaAnterior=" + fechaAnterior  
            + ", nuevaFecha=" + nuevaFecha  
            + "];"  
    }  
}
```

```
Prestamo.java
1  import java.time.LocalDate;
2  import java.util.ArrayList;
3  import java.util.List;
4
5
6  public class Prestamo {
7
8      private Estudiante estudiante;
9      private Ejemplar ejemplar;
10
11     private LocalDate fechaPrestamo;
12     private LocalDate fechaPrevistaDevolucion;
13     private LocalDate fechaRealDevolucion;    // null mientras no se devuelva
14
15     // Composicion: Prestamo es dueño del ciclo de vida de sus Renovaciones
16     private List<Renovacion> renovaciones = new ArrayList<>();
17
18
19     public Prestamo(
20         Estudiante estudiante,
21         Ejemplar ejemplar,
22         LocalDate fechaPrestamo,
23         LocalDate fechaPrevistaDevolucion) {
24
25         this.estudiante = estudiante;
26         this.ejemplar = ejemplar;
27         this.fechaPrestamo = fechaPrestamo;
28         this.fechaPrevistaDevolucion = fechaPrevistaDevolucion;
29         this.fechaRealDevolucion = null;
30
31         // El ejemplar pasa a no disponible al momento del prestamo
32         this.ejemplar.prestar();
33     }
34
35
36     public void renovar(LocalDate nuevaFecha) {
37         // Paso 1: validar que la nueva fecha sea posterior a la vigente
38         if (!nuevaFecha.isAfter(fechaPrevistaDevolucion)) {
39             throw new IllegalArgumentException(
```

```
39         throw new IllegalArgumentException(  
40             "La nueva fecha de devolucion (" + nuevaFecha  
41             + ") debe ser posterior a la fecha prevista vigente (" +  
42             fechaPrevistaDevolucion + ")"  
43         );  
44     }  
45  
46     // Paso 2: crear la Renovacion con los datos actuales  
47     Renovacion r = new Renovacion(  
48         LocalDate.now(),           // fecha en que se realiza la re  
49         fechaPrevistaDevolucion,   // fecha anterior  
50         nuevaFecha                 // nueva fecha  
51     );  
52  
53     // Paso 3: almacenar en el historial  
54     renovaciones.add(r);  
55  
56     // Paso 4: actualizar la fecha prevista  
57     fechaPrevistaDevolucion = nuevaFecha;  
58 }  
59  
60 /**  
61  * Devuelve true si el ejemplar no ha sido devuelto  
62  * y la fecha actual supera la fecha prevista (R08).  
63  */  
64 public boolean estaVencido(LocalDate hoy) {  
65     return fechaRealDevolucion == null  
66         && hoy.isAfter(fechaPrevistaDevolucion);  
67 }  
68  
69 /**  
70  * Registra la devolucion real del ejemplar (R09).  
71  * Valida que la fecha no sea anterior al prestamo.  
72  */  
73 public void registrarDevolucion(LocalDate fecha) {  
74     if (fecha.isBefore(fechaPrestamo)) {
```

```
73     public void registrarDevolucion(LocalDate fecha) {
74         if (fecha.isBefore(fechaPrestamo)) {
75             throw new IllegalArgumentException(
76                 "La fecha de devolucion no puede ser anterior a la fecha del prestamo"
77             );
78         }
79         this.fechaRealDevolucion = fecha;
80         this.ejemplar.devolver();
81     }
82
83     // Getters
84     public Estudiante getEstudiante() { return estudiante; }
85     public Ejemplar getEjemplar() { return ejemplar; }
86     public LocalDate getFechaPrestamo() { return fechaPrestamo; }
87     public LocalDate getFechaPrevistaDevolucion() { return fechaPrevistaDevolucion; }
88     public LocalDate getFechaRealDevolucion() { return fechaRealDevolucion; }
89     public List<Renovacion> getRenovaciones() { return renovaciones; }
90 }
91
```

5.3 Prueba mínima

```
Prestamo creado para: Santiago Parra
Ejemplar disponible despues del prestamo: false
Fecha prevista de devolucion inicial: 2026-09-15

--- PRUEBA 1: Renovacion valida ---
Prestamo renovado exitosamente.
Nueva fecha de devolucion: 2026-10-15
Cantidad de renovaciones: 1
Detalle: Renovacion[realizada=2026-09-30, fechaAnterior=2026-09-15, nuevaFecha=2026-10-15]
[NOTIFICACION para Santiago Parra] Su prestamo del ejemplar EJ-001 fue renovado hasta 2026-10-15
```

5.4 Prueba de renovación inválida

```
--- PRUEBA 2: Renovacion invalida ---
Intento: renovar con fecha 2026-09-20
(fecha prevista vigente: 2026-10-15)
Excepcion capturada correctamente: La nueva fecha de devolucion (2026-09-20) debe ser posterior a la fecha prevista vigente (2026-10-15)
El objeto Prestamo queda intacto. Renovaciones: 1

--- PRUEBA 3: Verificacion de herencia y roles ---
Estudiante es Usuario: true
Estudiante es Notificable: true
Bibliotecario es Usuario: true
Bibliotecario es Notificable: false

--- PRUEBA 4: Devolucion del ejemplar ---
Ejemplar disponible despues de devolver: true
[NOTIFICACION para Santiago Parra] Devolucion del ejemplar EJ-001 registrada correctamente.
PS C:\Users\Santiago\Downloads\SmartLibrary_Bloque5_Java>
```


6. Actividad 6 – Vista funcional mediante componentes UML

Objetivo. Construir una vista funcional introductoria de SmartLibrary mediante cuatro componentes: Gestión de Usuarios, Gestión de Catálogo, Gestión de Préstamos y Gestión de Reservas.

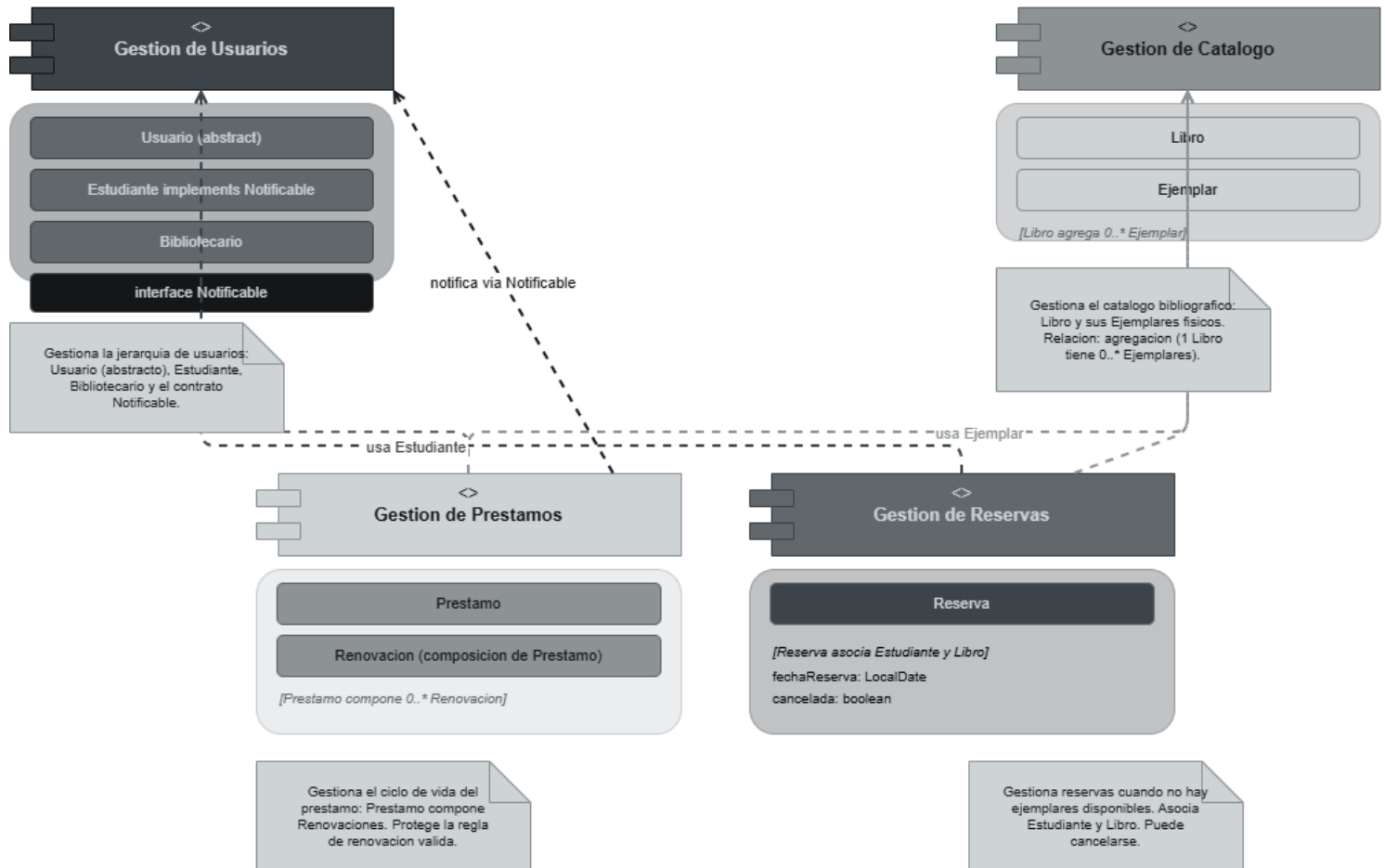
6.1 Distribución funcional

Componente	Clases relacionadas	¿De qué otro componente necesita información? ¿Para qué?
Gestión de Usuarios	Usuario, Estudiante, Bibliotecario	Puede aportar identificación y datos del usuario a Gestión de Préstamos y Gestión de Reservas para asociar operaciones con una persona concreta.
Gestión de Catálogo	Libro, Ejemplar	Gestión de Préstamos necesita información del ejemplar disponible; Gestión de Reservas necesita información del recurso que se desea reservar.
Gestión de Préstamos	Prestamo, Renovacion	Necesita información de Gestión de Usuarios para identificar al estudiante y de Gestión de Catálogo para identificar el ejemplar asociado al préstamo.
Gestión de Reservas	Reserva	Necesita información de Gestión de Usuarios para identificar al usuario y de Gestión de Catálogo para determinar el libro o ejemplar relacionado con la reserva.

Cuadro 4: Organización funcional de SmartLibrary.

6.2 Diagrama de componentes

Vista Funcional — Diagrama de Componentes SmartLibrary (Bloque 5)



7. Cierre y conclusión

Conclusión

Cuando dejamos de mirar las clases de manera aislada y empezamos a diseñar sus relaciones y contratos, el modelo adquiere un significado estructural más claro. Las relaciones permiten expresar cómo colaboran los objetos y cuándo existe una relación todo–parte; la herencia permite representar especializaciones que realmente son tipos de una abstracción común; y las interfaces permiten definir capacidades mediante contratos de comportamiento. En SmartLibrary, estas decisiones ayudan a representar reglas del dominio con mayor precisión y evitan elegir relaciones UML únicamente por conveniencia sintáctica. El resultado es un diseño en el que las clases no solo contienen datos, sino que participan en relaciones y responsabilidades coherentes con los requisitos del sistema.